

Exception Handling Tutorial

Exception Handling in Python

1. What is an Exception?

An exception is an error that occurs during program execution.

```
num = int(input('Enter a number: '))  
print(10 / num)
```

```
'''  
When Python encounters an error:  
  
    It stops normal execution  
    Raises an exception object  
    If not handled → program crashes  
'''
```

```
#Example:  
#1.Reading dataset file  
#2.Model loading  
#3.API response parsing  
#4.Database connections
```

```
#Types of Errors in Python:
```

```
#1. Syntax Error:Detected before execution.
```

```
#2. Logical Errors:Program runs but gives wrong output.
```

```
#3.Runtime Errors (Exceptions) Occur during execution.
```

```
....
```

```
Examples:
```

```
ZeroDivisionError
```

```
ValueError
```

```
TypeError
```

```
FileNotFoundError
```

```
IndexError
```

```
KeyError
```

```
....
```

2. Basic try-except Block

```
#Syntax:
```

```
try:
```

```
    # Code that may cause error
```

```
except:
```

```
    # Code to handle error
```

```
if:
```

```
    num = int(input('Enter number: '))
```

```
    print(10 / num)
```

```
else:
```

```
    print('An error occurred!')
```

```
print("After Try Except")
```

3.Handling Specific Exceptions

```
#Always catch specific exceptions instead of general ones.
```

```

try:
    num = int(input('Enter number: '))
    print(10 / num)
except ZeroDivisionError:
    print('You cannot divide by zero.')
except ValueError:
    print('Invalid input! Please enter a number.')

```

4. Using else and finally

```

#else runs only if no exception occurs.
#finally always runs.

```

```

try:
    num = int(input('Enter number: '))
    result = 10 / num
except ZeroDivisionError:
    print('Division by zero not allowed.')
except ValueError:
    print('Invalid input.')
else:
    print('Result:', result)
print("After the exception")

```

```

try:
    file = open("data.txt", "r")
    content = file.read()

except FileNotFoundError:
    print("File not found.")

finally:
    print("Execution completed.")

```

```

#finally mostly used for:
#   Closing files
#   Closing database connections
#   Releasing resources

```

5.Raising Exceptions

```
#You can manually raise exceptions.
```

```
age = int(input('Enter age: '))
if age < 0:
    raise ValueError('Age cannot be negative!')
print("After Exception")
```

```

'''
| `raise`                                | `try-except`                                |
| -----                                | -----                                |
| Used to throw an exception          | Used to handle an exception          |
| Creates an error                      | Catches an error                      |
| Stops normal flow                    | Controls error flow                    |
'''

```

```
'''
| Exception          | When It Occurs          |
| -----          | -----          |
| ZeroDivisionError  | Division by zero        |
| ValueError          | Wrong value type        |
| TypeError           | Wrong data type         |
| IndexError          | Invalid list index      |
| KeyError            | Invalid dictionary key  |
| FileNotFoundError   | File not found          |
| ImportError          | Module import fails     |
'''
```

6. Custom Exceptions

```
class InvalidMarksError(Exception):
    pass

marks = int(input('Enter marks: '))
if marks > 100:
    raise InvalidMarksError('Marks cannot exceed 100')
```

```
Enter marks: 200
InvalidMarksError: Marks cannot exceed 100
```

```
class ModelNotTrainedError(Exception):  
    pass
```

```
class AIModel:  
    def __init__(self):  
        self.trained = False  
  
    def train(self):  
        self.trained = True  
  
    def predict(self):  
        if not self.trained:  
            raise ModelNotTrainedError("Train the model before prediction.")  
        return "Prediction Result"
```

```
model = AIModel()  
  
try:  
    print(model.predict())  
except ModelNotTrainedError as e:  
    print("Error:", e)
```

Error: Train the model before prediction.

7. Using Exceptions inside functions

```
def calculate_average(marks_list):  
    try:  
        total = sum(marks_list)  
        avg = total / len(marks_list)  
    except ZeroDivisionError:  
        print('Marks list is empty.')  
        return None  
    except TypeError:  
        print('Marks must be numbers.')  
        return None  
    else:  
        return avg  
  
print(calculate_average([80, 90, 100]))
```